

Bases de Datos 1 — Caso de Estudio

SQL Programable

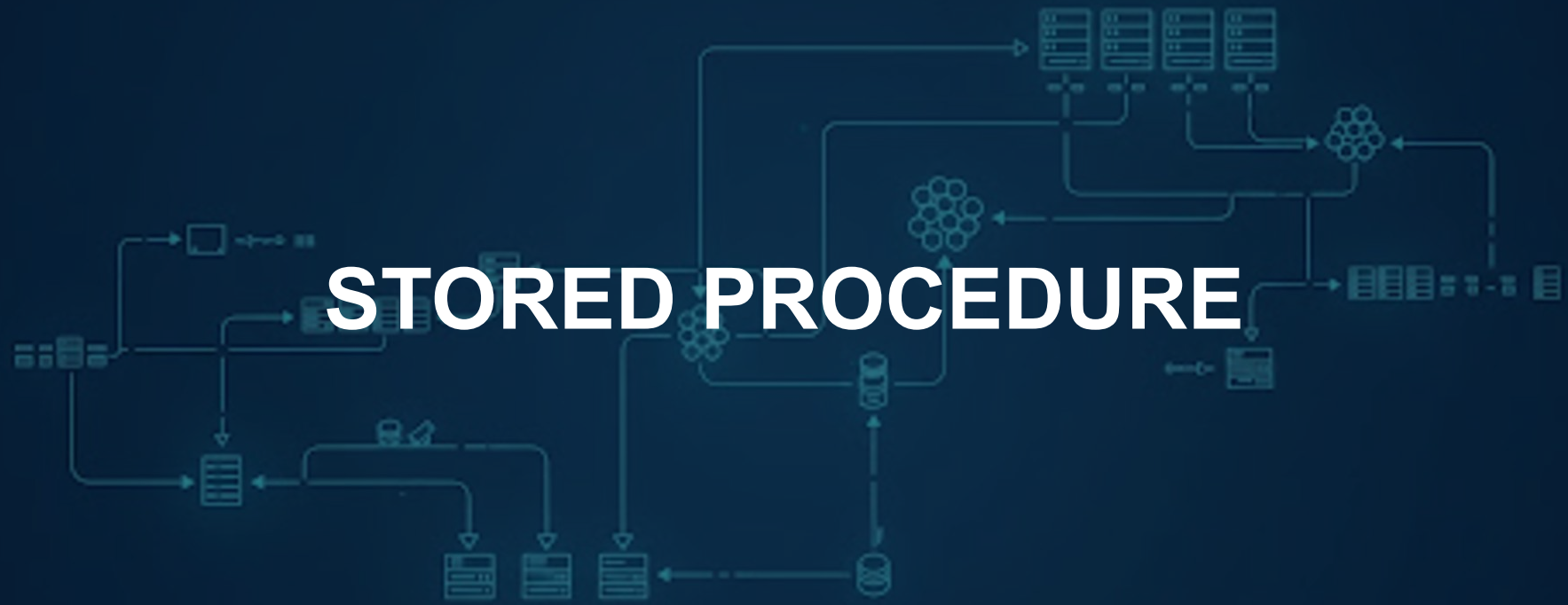
Procedimientos almacenados y Funciones



Bases de Datos 1

Equipo docente: Javier Vescio, Rocío Bustos y Franco Borsani.

STORED PROCEDURE



Bases de Datos 1

Equipo docente: Javier Vescio, Rocío Bustos y Franco Borsani.

Procedimientos Almacenados (SP)

¿Qué es un Stored Procedure?

- Un procedimiento almacenado es una rutina programable persistente, definida dentro de un SGBD, que encapsula una secuencia de operaciones sobre la base de datos y puede ser invocada mediante un nombre, permitiendo parametrización, control de flujo, manipulación de datos y, cuando corresponde, gestión de transacciones y errores.

Características

- ✓ Almacenamiento
- ✓ Ejecución
- ✓ Modularidad
- ✓ Reutilización
- ✓ Encapsulamiento
- ✓ Seguridad
- ✓ Validación

¿Cuándo utilizar una consulta SQL y cuándo un procedimiento almacenado?

STORED PROCEDURE

Sintaxis

- Crear SP

```
DELIMITER //
```

```
CREATE PROCEDURE sp_nombre_procedimiento (IN parametroEntrada INT, OUT parametroSalida INT)
```

```
BEGIN
```

```
    -- Cuerpo del procedimiento
```

```
END;
```

```
//
```

```
DELIMITER ;
```

CREATE PROCEDURE no ejecuta el procedimiento. Lo almacena.

- Ejecutar SP

```
CALL nombre_procedimiento();
```

Delimiter

Instrucción para indicar cómo detectar el final de la sentencia. Normalmente MySQL utiliza ; para indicar el final de una instrucción. Pero un procedimiento contiene varias instrucciones, por lo cual, para que MySQL no interprete el primer ; como el final del CREATE PROCEDURE, temporalmente cambiamos el delimitador.

Parámetros

Gestionan el flujo de datos de forma flexible:

- **IN (Entrada):** Envía valores para procesar la lógica.
- **OUT (Salida):** Devuelve valores calculados al finalizar.
- **INOUT:** Recibe, modifica y retorna un valor.

STORED PROCEDURE

- El procedimiento se almacena en el propio servidor MySQL y queda asociado a una base de datos, no simplemente al archivo SQL que utilizamos para crearlo.



STORED PROCEDURE

- Visualizar procedimientos existentes

Mostrar información sobre los procedimientos almacenados disponibles.

```
SHOW PROCEDURE STATUS;
```

Ver procedimientos creados en determinada base de datos:

```
SHOW PROCEDURE STATUS  
WHERE db = 'nombre_base_datos';
```

STORED PROCEDURE

- Visualizar procedimientos existentes

Ver un procedimiento en particular junto con su definición:

```
SHOW CREATE PROCEDURE nombre_procedimiento;
```

Procedimientos Almacenados (SP)

Ejemplo sin parámetros

```
DELIMITER //
```

```
CREATE PROCEDURE listar_estudiantes()  
BEGIN  
  
    SELECT *  
    FROM estudiante;  
  
END //
```

```
DELIMITER ;
```

```
CALL listar_estudiantes();
```

Procedimientos Almacenados (SP)

Ejemplo con parámetros de entrada

```
DELIMITER //
```

```
CREATE PROCEDURE listar_estudiantes_carrera(IN p_id_carrera INT)
```

```
BEGIN
```

```
    SELECT e.id_estudiante, e.nombre, e.apellido, e.email, c.nombre_carrera, i.fecha_inscripcion
```

```
    FROM estudiante e
```

```
    INNER JOIN inscripcion i ON e.id_estudiante = i.id_estudiante
```

```
    INNER JOIN carrera c ON i.id_carrera = c.id_carrera
```

```
    WHERE c.id_carrera = p_id_carrera
```

```
    ORDER BY e.apellido, e.nombre;
```

```
END //
```

```
DELIMITER ;
```

```
CALL listar_estudiantes_carrera(2);
```

Procedimientos Almacenados (SP)

Ejemplo con parámetros de salida

```
DELIMITER //  
CREATE PROCEDURE obtener_cantidad_estudiantes(OUT p_cantidad INT)  
BEGIN  
    SELECT COUNT(*)  
    INTO p_cantidad  
    FROM estudiante;  
END //  
DELIMITER ;
```

```
SET @cantidad = 0;  
CALL obtener_cantidad_estudiantes(@cantidad);  
SELECT @cantidad;
```

Procedimientos Almacenados (SP)

Ejemplo con parámetros de entrada y salida

```
DELIMITER //
```

```
CREATE PROCEDURE obtener_materias_aprobadas(IN p_id_estudiante INT,OUT p_cantidad_aprobadas INT)
```

```
BEGIN
```

```
    SELECT COUNT(*)
```

```
    INTO p_cantidad_aprobadas
```

```
    FROM cursada
```

```
    WHERE id_estudiante = p_id_estudiante AND nota >= 4;
```

```
END //
```

```
DELIMITER ;
```

```
SET @cantidad = 0;
```

```
CALL obtener_materias_aprobadas(1, @cantidad);
```

```
SELECT @cantidad;
```

Procedimientos Almacenados (SP)

Ejemplo con parámetro de entrada y salida

```
DELIMITER //
```

```
CREATE PROCEDURE validar_nota_final(INOUT p_nota DECIMAL(4,2))
```

```
BEGIN
```

```
    IF p_nota < 0 THEN
```

```
        SET p_nota = 0;
```

```
    ELSEIF p_nota > 10 THEN
```

```
        SET p_nota = 10;
```

```
    END IF;
```

```
END //
```

```
DELIMITER ;
```

```
SET @nota = 8.50;
```

```
CALL validar_nota_final(@nota);
```

```
SELECT @nota;
```

Variables

Una variable es un espacio de almacenamiento temporal identificado mediante un nombre, asociado a un tipo de dato y destinado a contener un valor durante la ejecución de una sentencia, rutina o sesión.

- **VARIABLES LOCALES:** Se declaran dentro de un bloque BEGIN...END mediante DECLARE. Su alcance está limitado al bloque donde fueron declaradas.

```
DECLARE contador INT DEFAULT 0;
```

- **VARIABLES DE USUARIO:** pertenecen a una sesión. Se pueden utilizar entre distintas sentencias dentro de una misma sesión. Son variables que comienzan con @ , por ejemplo, @nombre.

```
SET @nombre = 'Juan';
```

- **VARIABLES DEL SISTEMA:** almacenan valores de configuración o estado del servidor. Pueden ser de sesión (SESSION, conexión actual) o globales.

```
@@GLOBAL.nombre
```

```
@@SESSION.nombre
```

Variables en procedimientos almacenados

- Declarar una variable dentro del procedimiento:

```
DECLARE nombre_variable TIPO_DE_DATO  
[DEFAULT valor];
```

- Asignar valor:

```
SET nombre_variable = valor;
```

Ejemplo

```
CREATE PROCEDURE ejemplo()  
BEGIN  
  
    DECLARE v_total DECIMAL(10,2);  
  
    SET v_total = 100.50;  
  
    SELECT v_total;  
  
END;
```

Procedimientos Almacenados (SP)

Los procedimientos almacenados en MySQL pueden ejecutar prácticamente cualquier tipo de consulta SQL que se pueda ejecutar de forma directa en una instrucción SQL estandar. Esto incluye:

- Consultas de manipulación de datos:
 - SELECT: Recuperar datos de una o más tablas.
 - INSERT: Insertar datos en una tabla.
 - UPDATE: Actualizar datos en una tabla.
 - DELETE: Eliminar datos de una tabla.
- Consultas de control de flujo: esto incluye IF, ELSEIF, ELSE, CASE, bucles WHILE, REPEAT, etc.
- Llamadas a otros procedimientos almacenados:
Se pueden llamar a otros SP desde dentro de un SP.

Ejemplo declaración y asignación de variables y llamada a otro procedimiento almacenado

```
DELIMITER //
```

```
CREATE PROCEDURE registrar_cursada(IN p_id_estudiante INT,IN p_id_materia INT,IN p_anio YEAR,IN p_nota DECIMAL(4,2))
```

```
BEGIN
```

```
    -- Variable que será modificada por el procedimiento validar_nota_final
```

```
    DECLARE v_nota DECIMAL(4,2);
```

```
    -- Asignamos el valor de la nota recibida
```

```
    SET v_nota = p_nota;
```

```
    -- Llamamos al procedimiento que valida la nota
```

```
    CALL validar_nota_final(v_nota);
```

```
    -- Registramos la cursada con la nota validada
```

```
    INSERT INTO cursada (id_estudiante,id_materia,año,nota)
```

```
    VALUES (p_id_estudiante,p_id_materia,p_anio,v_nota);
```

```
END //
```

```
DELIMITER ;
```

STORED PROCEDURE

¿Cómo modificar un Stored Procedure?

MySQL no tiene una instrucción ALTER PROCEDURE para modificar el cuerpo del procedimiento. La estrategia habitual para cambiar la definición es:

```
DROP PROCEDURE nombre_procedimiento;
```

○

```
DROP PROCEDURE IF EXISTS nombre_procedimiento;
```

y posteriormente:

```
CREATE PROCEDURE nombre_procedimiento()
```

FUNCTIONS

Franco Borsani, Rocío Bustos, Javier Vescio.

Functions (funciones)

¿Qué es una función?

Una función almacenada es un programa almacenado en el servidor de base de datos que recibe parámetros de entrada, ejecuta una lógica determinada y retorna un único valor de un tipo de dato definido.

Principalmente sirven para encapsular lógica reutilizable que produce un valor.

Sintaxis:

```
DELIMITER //  
CREATE FUNCTION nombre_funcion(parametro TIPO_DE_DATO)  
RETURNS TIPO_RETORNO  
[NOT] DETERMINISTIC  
BEGIN  
    -- Instrucciones SQL  
    RETURN valor;  
END //  
DELIMITER ;
```

FUNCTIONS

Ejemplo:

```
DELIMITER //
```

```
CREATE FUNCTION promedio_estudiante(p_id_estudiante INT)
```

```
RETURNS DECIMAL(4,2)
```

```
READS SQL DATA
```

```
BEGIN
```

```
    DECLARE v_promedio DECIMAL(4,2);
```

```
    SELECT AVG(nota)
```

```
    INTO v_promedio
```

```
    FROM cursada
```

```
    WHERE id_estudiante = p_id_estudiante;
```

```
    RETURN v_promedio;
```

```
END //
```

```
DELIMITER ;
```

¿Cómo se llaman/ejecutan?

Una función, a diferencia de un procedimiento almacenado, se puede utilizar dentro de una expresión SQL. Se comporta conceptualmente como las funciones incorporadas de MySQL.

```
SELECT id_estudiante, nombre, apellido, promedio_estudiante(id_estudiante) AS promedio
FROM estudiante;
```

```
SELECT nombre, apellido, promedio_estudiante(id_estudiante) AS promedio
FROM estudiante
WHERE promedio_estudiante(id_estudiante) >= 7;
```

FUNCTIONS

DETERMINISTIC

Dados los mismos parámetros de entrada, la función produce el mismo resultado.

```
DELIMITER //
CREATE FUNCTION nombre_completo(p_nombre VARCHAR(50),p_apellido VARCHAR(50))
RETURNS VARCHAR(101)
DETERMINISTIC
BEGIN
    RETURN CONCAT(p_apellido, ' ', p_nombre);
END //
DELIMITER ;
```

NOT DETERMINISTIC

Puede devolver resultados distintos aún con los mismos parámetros.

```
DELIMITER //
CREATE FUNCTION obtener_fecha_actual()
RETURNS DATE
NOT DETERMINISTIC
BEGIN
    RETURN CURDATE();
END;
//
DELIMITER ;
```

Diferencia e/ procedimientos almacenados y funciones

Característica	Stored Procedure	Función
Valor de retorno	No devuelve un valor directamente (usa OUT)	Devuelve un único valor con RETURN
Llamada	Se invoca con CALL nombreProcedimiento()	Se invoca como parte de una expresión (SELECT)
Uso en SQL (SELECT, WHERE, etc.)	No se puede usar	Sí se puede usar
Cantidad de valores de salida	Múltiples, usando parámetros OUT	Uno solo, con RETURN
Tipo de parámetros	IN, OUT, INOUT	Solo IN
Modificación de datos	Sí (puede usar INSERT, UPDATE, etc.)	No recomendada